



UNIVERSIDAD
CAECE

Cámara Argentina de Comercio y Servicios

TECNICATURA EN PROGRAMACIÓN

BASES DE DATOS

GUIA DE LA UNIDAD Nro. 4

TÉCNICAS DE MANIPULACIÓN DE DATOS

Contenidista: Lic. Pablo R. Fernández

ÍNDICE – UNIDAD 4:

INTRODUCCIÓN	5
1. ÁLGEBRA RELACIONAL	5
1.1 CÁLCULO RELACIONAL	5
2. OPERACIONES DEL ÁLGEBRA RELACIONAL	6
2.1 LA OPERACIÓN JOIN	8
ACTIVIDAD 1	10
3. FORMAS NORMALES	10
ACTIVIDAD 2	12
4. XML - EXTENSIBLE MARKUP LANGUAGE	13
ACTIVIDAD 3	17
5. TÉCNICAS DE IMPORTACIÓN	16
5.1 UTILIZAR TÉCNICAS DE IMPORTACIÓN PARA ARCHIVOS DE TEXTO	18
AUTOEVALUACIÓN	19
SÍNTESIS DE LA UNIDAD	20

MAPA DE LA UNIDAD 4

PROPÓSITOS

En esta unidad nos proponemos explicarle los principales comandos del lenguaje de consultas SQL como así también algunos conceptos más avanzados tales como las vistas, los stored procedures y los triggers. También veremos cómo se maneja la seguridad y se administran los permisos sobre los objetos de la base de datos.

OBJETIVOS

- ✓ Analizar las Operaciones del álgebra relacional
- ✓ Conocer y aplicar las formas normales
- ✓ Utilizar lenguaje XML para realizar importación/exportación de datos.
- ✓ Utilizar mecanismos de importación y exportación de diferentes motores de base de datos
- ✓ Realizar intercambio de datos utilizando archivos planos

CONTENIDOS

Para que alcance los objetivos, los contenidos que abordará son los siguientes:

- 1) Álgebra relacional y cálculo relacional.
- 2) Operaciones del álgebra relacional: operación de selección, proyección, reunión, y operaciones de conjuntos: Unión, resta e intersección.
- 3) Formas normales.
- 4) Uso de XML: archivo dtds y schemas
- 5) Técnicas de importación y exportación implementadas por los distintos motores.
- 6) Intercambio de información a través de archivos planos.

PALABRAS CLAVES

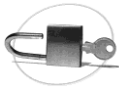
Álgebra relacional, formas normales, importación, exportación, XML.



BIBLIOGRAFÍA

Elmasri, Ramez/Navathe, Shamkant (2011). *Fundamentos de Sistemas de Base de Datos*. 6ta Ed, EEUU: Pearson / Addison Wesley.

INTRODUCCIÓN



ABORDEMOS EL LOGRO DEL PRIMER OBJETIVO DE LA UNIDAD:

- ✓ Analizar las Operaciones del álgebra relacional

Para comenzar con esta unidad, y continuando desde las Unidades 4 y 5, donde ya estuvimos interactuando con la base de datos utilizando el lenguaje SQL, ahora veremos esas mismas operaciones pero expresadas en la forma pensada para crear estructuras de álgebra relacional. Además aprenderemos a usar lenguaje XML para interactuar con datos externos.

Comencemos...



1. ÁLGEBRA RELACIONAL

Las operaciones de álgebra relacional que veremos a continuación, nacieron antes que el lenguaje SQL y son justamente sus precursoras. Estas producen nuevas relaciones, que se puede manipular aún más usando operaciones del mismo álgebra. Una secuencia de operaciones de álgebra relacional forma una expresión de álgebra relacional, cuyo resultado también será una relación que representa el resultado de una consulta de base de datos



Recordarás que hablamos de cómo el lenguaje SQLVpodía ayudarnos a interactuar con las t-uplas de las relaciones de la base de datos. Pues, el lenguaje SQL está basado en el álgebra relacional.

La lógica utilizada para implementar consultas en el módulo de optimización de consultas del RDBMS se basa en técnicas nacidas del álgebra relacional. Las operaciones centrales y las funciones en los módulos internos de la mayoría de los sistemas relacionales también se basan en operaciones de álgebra relacional.

1.1 Cálculo relacional

El cálculo relacional proporciona un lenguaje declarativo de nivel superior para especificar consultas relacionales. En una expresión de cálculo relacional, no hay un orden específico

para las operaciones ni para definir cómo recuperar la consulta, solo existe el resultado que esta arroja.

El cálculo relacional es importante porque se basa en lógica matemática y porque el lenguaje de consulta estándar (SQL) para RDBMS tiene algunos de sus fundamentos en una variación de cálculo relacional conocida como cálculo relacional de tupla.

El álgebra relacional a menudo se considera una parte integral del modelo de datos relacional. Sus operaciones se pueden dividir en dos grupos. Un grupo incluye operaciones de conjunto de la teoría de conjuntos matemáticos: UNIÓN, INTERSECCIÓN, RESTA DE CONJUNTOS y PRODUCTO CARTESIANO. El otro grupo consiste en operaciones desarrolladas específicamente para trabajar con la t-uplas de las bases de datos relacionales. Estas operaciones son SELECT, PROJECT, y JOIN.



ABORDEMOS EL LOGRO DEL SEGUNDO OBJETIVO DE LA UNIDAD:

- ✓ Analizar las Operaciones del álgebra relacional



2. OPERACIONES DEL ÁLGEBRA RELACIONAL.

La operación SELECT

Esta operación se denota con la letra griega σ (Sigma) y se usa para elegir un subconjunto de las tuplas de una relación que satisface una condición de selección.

Es decir que podríamos considerar la operación SELECT como un filtro que solo considerará aquellas tuplas que satisfacen una condición calificada.

O sea que esta operación es como una partición horizontal de la relación en dos conjuntos de tuplas: las tuplas que satisfacen la condición se seleccionan, y las tuplas que no satisfacen la condición quedan afuera.

Veamos un ejemplo utilizando el modelo relacional de la práctica 3:

Seleccionar los empleados cuyo sueldo es mayor a 30000\$ y que pertenecen al departamento de Ventas.

$\sigma_{\text{Salario} > 30000 \text{ and Cod_Dep} = \text{'Ventas'}}(\text{EMPLEADO})$

Esta operación es equivalente a la siguiente sentencia SQL:

```
select * from <Tabla> where <lista de condiciones>
```

Donde es importante distinguir que siempre se seleccionará la t-upla completa (o sea todos los atributos de la relación), pero se filtrarán t-uplas con la condición.

Además, es importante destacar que la operación SELECT es conmutativa, es decir:

$$\sigma_{\langle \text{condición1} \rangle}(\sigma_{\langle \text{condición2} \rangle}(\text{Tabla})) = \sigma_{\langle \text{condición2} \rangle}(\sigma_{\langle \text{condición1} \rangle}(\text{Tabla}))$$

La operación PROJECT

La operación PROYECTO, selecciona ciertas columnas de la tabla y descarta las otras columnas, siempre tomando todas las t-uplas de la relación. Si nos interesan solo ciertos atributos de una relación, usaremos entonces la operación PROYECTO, denotada por la letra griega π (pi) para proyectar la relación sobre estos atributos solamente. Por lo tanto, el resultado del PROYECTO se puede visualizar como una partición vertical de la relación en dos relaciones.

Por ejemplo si queremos tener la lista de los nombres de los empleados (no todos sus datos) haremos:

$\pi_{\text{Lname, Fname}}(\text{EMPLEADO})$

Generalizando, podemos expresar la operación PROJECT como:

$\pi_{\langle \text{lista de atributos} \rangle}(\text{Tabla})$

Usted debe recordar que si la lista de atributos incluye solo atributos no clave de R, es posible que haya t-uplas repetidas en la tabla, pero la operación PROYECTO elimina cualquier tupla duplicada, por lo que el resultado de la operación PROYECTO es un conjunto de tuplas distintas.

Por lo tanto, la operación PROJECT es equivalente a la siguiente sentencia de SQL:

Select distinct <lista de campos> from <Tabla>

Además se cumple que:

$$\pi_{\langle \text{lista de atributos 1} \rangle}(\pi_{\langle \text{lista de atributos 2} \rangle}(\text{Tabla})) = \pi_{\langle \text{lista de atributos 1} \rangle}(\text{Tabla})$$

siempre que la Lista de atributos 1 contenga la lista de atributos 2. Es decir se aplica la proyección externa que sería la última. Si Lista de atributos 2 no contiene la lista de atributos 1 la expresión resultante arroja un error. Por lo tanto queda claro que la operación PROJECT no es conmutativa.

Las operaciones UNIÓN, INTERSECT y MINUS

Estas son operaciones binarias; es decir, cada uno se aplica a dos conjuntos (de tuplas). Cuando estas operaciones se aplican a conjuntos de relaciones de bases de datos, las dos relaciones en el que se aplica cualquiera de estas tres operaciones debe tener compatibilidad de unión o compatibilidad de tipo.

Se dice que dos relaciones R (A1, A2, ..., An) y S (B1, B2, ..., Bn) son unión compatible (o compatible con el tipo) si:

- 1) tienen el mismo grado n
- 2) si $\text{dom}(A_i) = \text{dom}(B_i)$ para $1 \leq i \leq n$.

Esto significa que las dos relaciones tienen el mismo número de atributos y cada par correspondiente de atributos tiene el mismo dominio.

Podemos definir las tres operaciones UNION, INTERSECT y MINUS aplicadas a dos relaciones R y S compatibles con la unión de la siguiente manera:

UNION: $R \cup S$, es una relación que incluye todas las tuplas distintas que están en R o en S o en R y S .

INTERSECT: $R \cap S$, es una relación que incluye todas las tuplas que están tanto en R como en S .

MINUS: $R - S$, es una relación que incluye todas las tuplas que están en R pero no en S .

Como se podrá imaginar, al igual que con las sentencias SQL, se pueden combinar las operaciones del álgebra relacional para generar expresiones bastante complejas.

Las operaciones UNION e INTERSECT son conmutativas; es decir que se cumple:

$$\begin{aligned}\text{UNION: } R \cup S &= S \cup R \\ \text{INTERSECT: } R \cap S &= S \cap R\end{aligned}$$

Es importante recordar que ambas son también operaciones asociativas, o sea:

$$\begin{aligned}\text{UNION: } R \cup (S \cup T) &= (R \cup S) \cup T \\ \text{INTERSECT: } (R \cap S) \cap T &= R \cap (S \cap T)\end{aligned}$$

Pero, la operación MINUS es no conmutativa, ya que

$$\text{MINUS: } R - S \neq S - R$$

NOTA: INTERSECT se puede expresar también como:

$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$

2.1 La operación JOIN

La operación JOIN, denotada por $\bowtie$, se usa para combinar tuplas relacionadas de dos relaciones en tuplas únicas. Por ejemplo cuando necesitemos el nombre del empleado y el nombre del departamento en el que trabaja tendremos que usar el join entre las relaciones Empleado y Departamento (usando su PK-FK para realizarlo) y poder traer dentro de la misma operación los atributos de ambas relaciones juntos. Luego si es necesario se puede aplicar una operación PROJECT para reducir a los atributos que se necesiten.

$\pi_{\langle LName, DName \rangle} (\text{Empleado} \bowtie \text{Departamento})$

Si los nombres de la PK y la FK que participan en el JOIN son iguales se trata de un JOIN natural y no es necesario colocarlos, si esto no es así, se deben poner debajo del signo los nombres de los atributos participantes de la operación de JOIN (por ej, $\text{Empleado.CodDep} = \text{Departamento.Cod_Dep}$).

En su forma más general, para realizar el JOIN entre $R(A_1, A_2, \dots, A_n)$ y $S(B_1, B_2, \dots, B_m)$ expresaríamos:

$R \bowtie \langle \text{condición de join} \rangle S = Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$

Q tiene una t-upla por cada condición que se satisface entre R y S siendo la t-upla compuesta por todos los atributos de R y luego todos los de S.

Todo lo que acabamos de ver parece muy abstracto y usted se preguntará cómo se puede aplicar a casos concretos.

Veamos entonces algunos ejemplos:



Si tuviéramos por ejemplo que buscar cuáles son los pedidos de los empleados que se realizaron en los últimos 3 meses, ¿Haríamos una selección o una proyección?

Como no se especifica qué campos hay que buscar (solo indica que se necesitan los pedidos), pero sabemos que hay una condición (los últimos 3 meses) entonces claramente es una operación de selección lo que debemos hacer:

$\sigma_{\text{mes}(\text{FechaPedido}) > \text{mes}(\text{hoy}) - 3}(\text{Pedidos})$

Si nos piden solamente los números de pedidos entonces, tenemos que saber que es una proyección (porque se restringen las columnas devueltas, no son todas las columnas de la selección. Entonces:

$\pi_{\text{Nro Pedido}}(\sigma_{\text{mes}(\text{FechaPedido}) > \text{mes}(\text{hoy}) - 3}(\text{Pedidos}))$

O sea como usted puede ver, hemos aplicado la proyección al resultado obtenido por la selección.

Si lo que nos pedían hubiera sido la lista de los códigos de productos no vendidos en esos tres meses, tendríamos que pensar primero en sacar la lista de los productos que sí se vendieron y a partir de la lista completa de productos, restar los que se vendieron para obtener aquellos que no.

Esta operación de resta es equivalente a lo que hacemos con un NOT IN en SQL, sin embargo es mejor desde el punto de vista de la performance, ya que la resta opera con conjuntos de datos en lugar de realizar las subconsultas anidadas. Por este motivo los motores de bases de datos agregaron las sentencias EXCEPT (en SQL Server) y MINUS (en Oracle).

Veamos el ejemplo en álgebra relacional:

$\pi_{\text{Nro Producto}}(\text{Pedidos}) - \pi_{\text{Nro Producto}}(\sigma_{\text{mes}(\text{FechaPedido}) > \text{mes}(\text{hoy}) - 3}(\text{Pedidos} \bowtie \text{DetallePedidos}))$



ACTIVIDAD 1

Le pedimos que a continuación realice la Actividad de autoevaluación que le proponemos en el campus para ir afianzando los conceptos vistos hasta aquí



ABORDEMOS EL LOGRO DEL TERCER OBJETIVO DE LA UNIDAD:

- ✓ Comprender y aplicar las formas normales



3. FORMAS NORMALES.

Ahora que ya hemos aprendido a armar las estructuras de datos para modelar una situación, estamos en el punto perfecto para aprender cómo deben definirse estas estructuras correctamente. Para eso, debemos adecuar nuestro modelado a ciertas condiciones que se denominan “formas normales”.

Las formas normales brindan una serie de reglas para dar contexto al diseño de bases de datos de manera que aseguren que éste sea lo más correcto posible. Estas reglas ayudan a los diseñadores a realizar su trabajo ya que las deben aplicar cuando arman los DER. Cabe aclarar que las formas normales son de estructura jerárquica, es decir, si un diseño no está en 1ra forma normal entonces no podrá estar tampoco en 2da o 3ra forma normal. Solamente veremos estas tres formas normales ya que son las formas básicas.

➤ Primera Forma Normal

Esta primera forma es muy simple y en realidad lo que indica es que todos los atributos que se manejen y utilicen en una base de datos deben ser simples o atómicos. Es decir, no deben ser divisibles en atributos más pequeños. Ejemplos típicos podrían ser el nombre de un

contacto, que debería estar separado en nombre y apellido, dos atributos en vez de uno. Otro ejemplo sería la dirección, que debería en realidad ser calle, número, piso, departamento, localidad, provincia y país, o si utilizamos bases de datos con atributos geográficos, se podría también agregar la latitud y la longitud. De todas maneras, obviamente no es solo un atributo “dirección” con todos esos datos incluidos en el mismo, sino varios atributos.

Si la base de datos tiene atributos de este tipo, NO está ni siquiera en primera forma normal.

➤ Segunda Forma Normal

Como vimos anteriormente, para decidir si un diseño está en segunda forma normal, primero debemos analizar si está en primera forma. Si no lo está, no tendrá sentido analizar si cumple con la segunda forma.

En una tabla, todos los atributos de la misma deben depender DIRECTAMENTE de la clave primaria. Esto quiere decir que no puede haber un atributo totalmente desconectado del resto de los atributos de la tabla, especialmente de su clave.

Un ejemplo sería si en la tabla empleados, cuya clave es el legajo, tuviéramos el departamento donde trabaja (por ejemplo “Compras”) ya sea el nombre o el código del departamento. Esto claramente violaría la 2da forma normal ya que el departamento no depende del legajo. Para solucionar esto, se debe separar el departamento junto con el *legajo* y armar una tabla nueva que se llame por ejemplo Empleados_Departamentos con ambos atributos como clave.

➤ Tercera Forma Normal

Esta forma normal establece que todas las claves no primarias de una relación no deben depender de otros atributos que tampoco son claves primarias. Es decir que no puede haber dependencias transitivas entre los atributos de una tabla (o relación, como se la llama en su forma más general). Por ejemplo, la tabla *Empleado* no podría contener como atributos al *Código del departamento* y al *Nombre del departamento* donde trabaja el empleado ya que la clave de la tabla empleado es *legajo*, el nombre del departamento depende del código del departamento (que no es clave en la tabla empleado). Para solucionar esto, obviamente se debe separar en una tabla aparte, la tabla Departamento, ambos atributos colocando como clave al *Código* y como atributo dependiente de la clave al *Nombre*. Quedan entonces en la tabla Empleado, todos atributos que dependen de la clave directamente y no queda ninguna dependencia transitiva. Con lo cual está en 1ra, 2da y 3ra forma normal.



ACTIVIDAD 2

Le pedimos que a continuación realice la Actividad de autoevaluación que le proponemos en el campus para ir afianzando los conceptos vistos hasta aquí



ABORDEMOS EL LOGRO DEL SEGUNDO OBJETIVO DE LA UNIDAD:

- ✓ Utilizar lenguaje XML para realizar importación/exportación de datos



4. XML - EXTENSIBLE MARKUP LANGUAGE

Usted habrá notado que muchas aplicaciones de Internet proporcionan interfaces web para acceder a la información almacenada en una o más bases de datos que son para esa aplicación web lo que se denomina fuente de datos.

Hay varios lenguajes para escribirlas interfaces web (las páginas web que usamos para interactuar con nuestras aplicaciones), el más común es HTML (HyperText Markup Language). Pero aunque HTML se use para formatear y estructurar documentos web, no es adecuado para especificar datos estructurados que se extraen de las bases de datos. Los datos extraídos de las bases de datos relacionales pueden ser formateados como documentos XML.

Un lenguaje más apropiado es el XML (lenguaje de marcado extensible) que es ya un standard para estructurar e intercambiar datos a través de la Web en archivos de texto. También se ha comenzado a utilizar JSON (JavaScript Object Notation) para el mismo propósito.

La ventaja es que XML se puede utilizar para proporcionar información sobre la estructura y significado de los datos en las páginas web en lugar de simplemente especificar cómo las páginas están formateadas para mostrarlas en la pantalla.

XML se puede utilizar para transferir información en archivos de texto autodescriptivos entre varios programas en diferentes computadoras cuando las aplicaciones lo necesiten.

El modelo de datos XML, se basa en estructuras de jerárquicas y posee lenguajes propios para especificar una estructura de documentos: el DTD (Document Type Definition) y XML Schema.

Una diferencia clave entre los datos estructurados y semiestructurados se refiere a cómo se manejan las construcciones de los esquemas (por ejemplo, los nombres de atributos, relaciones y tipos de entidad) ya que en datos semiestructurados, la información del esquema se mezcla con los datos, entonces cada objeto de datos puede tener diferentes atributos que no se conocen de antemano.

En la figura siguiente vemos un ejemplo de un código XML y cómo las etiquetas que representan los nombres de esquema: los nombres de los atributos, los tipos de objeto (o tipos de entidad o clases) y las relaciones se mezclan con los nodos internos que representan los atributos y los valores de datos.

Entonces podemos decir que, en el modelo semiestructurado:

- No hay ningún requisito para un esquema predefinido al que deben ajustarse los objetos de datos, aunque es posible predefinirlo si es necesario
- La información del esquema, nombres de atributos, relaciones y clases (tipos de objetos) se entremezcla con los objetos y sus valores de datos en la misma estructura.

Lo primero que observamos en la estructura del documento XML es que trae la versión en su primera línea.

Para construir un documento XML se utilizan dos conceptos principales de estructuración: elementos y atributos.

Los atributos en XML dan información adicional que describe los elementos. Además en XML hay entidades, identificadores y referencias y se utilizan tags que están encerrados entre signos < y >, y al igual que en otros lenguajes dichos tags se abren con <nombre tag> y se cierran con </ nombre tag >.

```
<?xml version="1.0" standalone="yes"?>
<Projects>
  <Project>
    <Name>ProductX</Name>
    <Number>1</Number>
    <Location>Bellaire</Location>
    <Dept_no>5</Dept_no>
    <Worker>
      <Ssn>123456789</Ssn>
      <Last_name>Smith</Last_name>
      <Hours>32.5</Hours>
    </Worker>
  </Project>
  <Project>
    <Name>ProductY</Name>
    <Number>2</Number>
    <Location>Sugarland</Location>
    <Dept_no>5</Dept_no>
    <Worker>
      <Ssn>123456789</Ssn>
      <Hours>7.5</Hours>
    </Worker>
    <Worker>
      <Ssn>453453453</Ssn>
      <Hours>20.0</Hours>
    </Worker>
    <Worker>
      <Ssn>333445555</Ssn>
      <Hours>10.0</Hours>
    </Worker>
  </Project>
  ...
</Projects>
```

Podemos también ver que existen elementos complejos que se construyen a partir de otros elementos jerárquicamente, mientras que los elementos más simples contienen solo los valores de datos.

Existe un documento opcional que se puede usar para guardar los nombres de la etiqueta XML (elemento), este documento lleva el nombre de **schema**.

En la figura podemos identificar los elementos simples ya que son los que tienen la etiqueta alrededor del dato, por ejemplo <Nombre>, <Número>, <Ubicación>, <Nombre del departamento>, <Ssn>, <Apellido>, <Primer nombre>, y <Horas>.

Los elementos complejos son los que tienen los nombres de etiqueta anidados con otros elementos. Por ejemplo <Proyectos>, <Proyecto> y <Trabajador>. No hay límite en los niveles de anidación de elementos.

Los motores de bases de datos han implementado el uso de XML para exportar e importar datos además de incluir el tipo XML para poder guardar código XML dentro de las bases de datos.

Las implementaciones en cada motor son diferentes, así que no son comandos ANSI, es necesario entonces revisar para cada DBMS cuales son los comandos apropiados antes de poder utilizarlos.



Se preguntará entonces cómo se puede obtener una lista de datos de una base de datos en formato XML, listo para poder ser utilizado.

Vamos a realizar el ejemplo utilizando SQL Server y la base master para obtener la lista de campos de la tabla . Por ejemplo, si se hace la siguiente consulta en lenguaje SQL:

```
SELECT id,uid,crdate,version
FROM sysobjects
where name like 'syso%'
```

Los primeros cuatro registros devueltos son los siguientes:

id	uid	crdate	version
-105	4	2016-04-30 00:44:25.880	0
94	4	2009-04-13 12:59:08.060	0
60	4	2009-04-13 12:59:05.687	0
-207	4	2016-04-30 00:44:27.490	0

Pero si quisiéramos obtener estos campos en formato XML, se hace de la siguiente forma:

```
SELECT id,uid,crdate,version
FROM sysobjects
where name like 'syso%'
FOR XML PATH;
```

Y el resultado equivalente a los datos obtenidos en la tabla de arriba, es el siguiente:

```
<row>
<id>-105</id>
<uid>4</uid>
<crdate>2016-04-30T00:44:25.880</crdate>
<version>0</version>
</row>
<row>
<id>94</id>
<uid>4</uid>
<crdate>2009-04-13T12:59:08.060</crdate>
<version>0</version>
</row>
<row>
<id>60</id>
<uid>4</uid>
```



```
<crdate>2009-04-13T12:59:05.687</crdate>
<version>0</version>
</row>
<row>
<id>-207</id>
<uid>4</uid>
<crdate>2016-04-30T00:44:27.490</crdate>
<version>0</version>
</row>
</row>
```



ACTIVIDAD 3

Le pedimos que a continuación realice la Actividad de autoevaluación que le proponemos en el campus para ir afianzando los conceptos vistos hasta aquí.



ABORDEMOS EL LOGRO DEL QUINTO OBJETIVO DE LA UNIDAD:

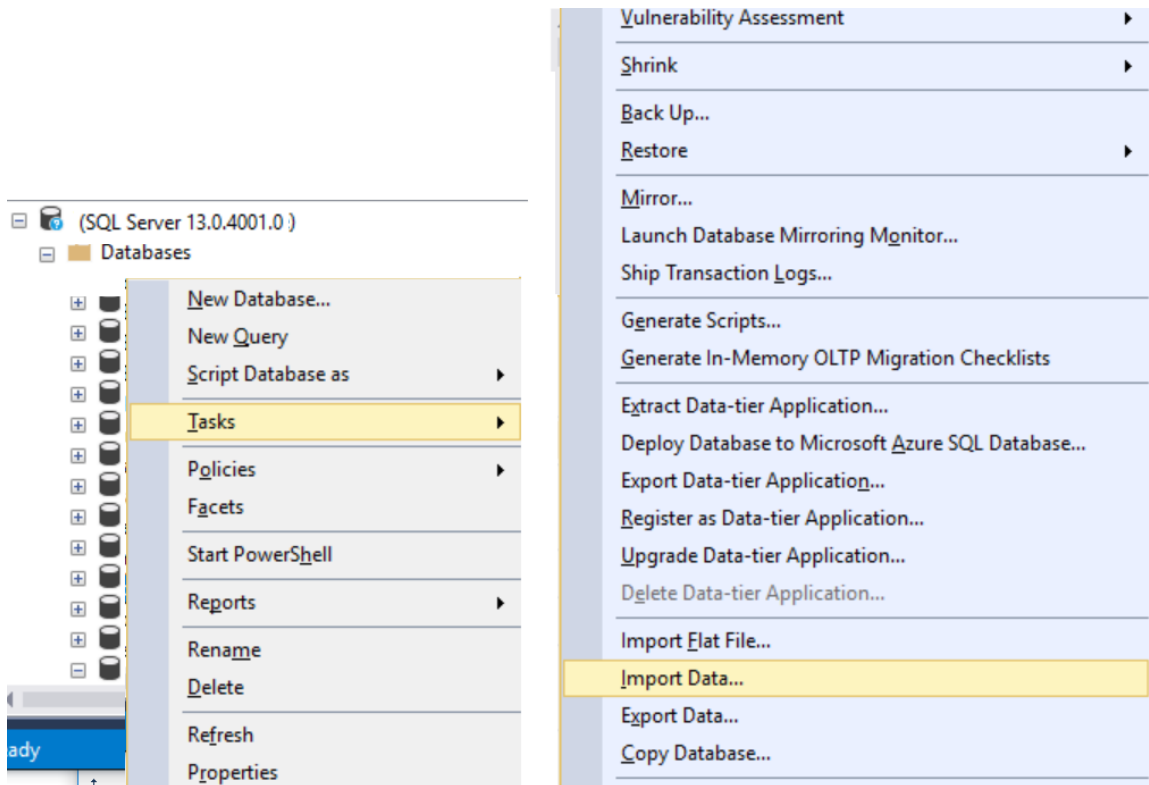
- ✓ Utilizar mecanismos de importación y exportación de diferentes motores de base de datos



5. TÉCNICAS DE IMPORTACIÓN

Los motores de base de datos incluyen módulos propios para ayudar a los usuarios a importar datos procedentes de diferentes orígenes. Estos módulos suelen ser aplicaciones bastante amigables que brindan un paso a paso simple que ayuda a lograr los objetivos básicos generales de importar datos desde orígenes comunes.

En el caso de SQL Server, partiendo de SQL Server Management Studio, haciendo click con el botón derecho sobre la base de datos de destino, eligiendo Tasks, se abre un menú desde donde se puede seleccionar Importar Datos.



ABORDEMOS EL LOGRO DEL ULTIMO OBJETIVO DE LA UNIDAD:

- ✓ Aprender a realizar intercambio de datos utilizando diferentes tipos de archivos



5.1 Utilizar técnicas de importación para archivos de texto

En el caso de archivos planos, que es el caso de los archivos de texto más habitual, se puede recurrir a las técnicas citadas en el apartado anterior, es decir, recurrir a los entornos gráficos que brindan los DBMS para realizar importación de datos, los cuales incluyen siempre algunos de los archivos planos más comunes (csv, tsv, txt). También, en el caso de los csv, tsv o txt se los puede abrir previamente con excel o cualquier planilla de cálculo y seguir el proceso explicado en el apartado anterior para ese caso.

Un problema muy común suelen ser los símbolos presentes en los archivos, sobre todo las comillas dobles o simples. Para estos casos, lo que suele hacerse es reemplazarlas con otro símbolo menos disruptivo en SQL. Las comillas en los textos provocan fallas en la importación y diversos tipos de errores pero quitándolas o reemplazándolas se solucionan la mayoría de los problemas.

Otro inconveniente muy típico es el de las fechas, ya que los formatos son muy específicos de la configuración de los DBMS y al importarlas se suelen transformar los formatos logrando resultados inesperados.

Para evitar este inconveniente, se suelen transformar las fechas a textos (tipo varchar en lenguaje SQL) y a continuación se realiza la transformación relacionada desde el DBMS una vez importados los datos exitosamente.



AUTOEVALUACIÓN

Realizar la autoevaluación propuesta.



Si comprendiste los conceptos descriptos en esta unidad, habrás logrado comprender las principales características del lenguaje SQL, el manejo de usuarios y permisos y la recuperación de las bases de datos.

Has logrado alcanzar los objetivos propuestos para esta Unidad.

¡FELICITACIONES!



SÍNTESIS DE LA UNIDAD

Hagamos un resumen de todo lo visto en esta Unidad.

Las SENTENCIAS del ÁLGEBRA RELACIONAL se corresponden con sentencias propias del LENGUAJE SQL y se utilizan para mejorar las técnicas del LENGUAJE DE CONSULTAS. Las SENTENCIAS SQL también incluyen CLÁUSULAS relacionadas con SENTENCIAS XML que permiten EXPORTAR DATOS desde BASES DE DATOS a ARCHIVOS XML para su posterior utilización. El LENGUAJE XML permite operar con datos en diversos ENTORNOS DE PROGRAMACIÓN con lo cual se facilita el manejo de los datos.

Muchas veces se necesita interactuar con datos desde diversas FUENTES DE DATOS tales como TEXTOS PLANOS, PLANILLAS DE CÁLCULO, ARCHIVOS XML y otros DBMS. Para poder realizar esa interacción es que se utilizan las TÉCNICAS DE IMPORTACIÓN y EXPORTACIÓN de datos.

